

kayfabe

A Mode-2 NVIDIA GPU forwarding stack for KVM/QEMU guests

An architecture whitepaper, written to be attacked

Subject. Two repositories on one machine. `/workspace/nvidia-gpu-passthrough` — the C research artifact, 739 commits, 44 031 lines of C, developed 2026-05-24 to 2026-07-24; it works on real hardware. `/workspace/nvkvm-rs` — **kayfabe**, a clean-slate Rust rewrite, 93 commits, 38 224 lines of implementation and 39 469 lines of test code, developed 2026-07-24 to 2026-07-27; it has never touched a GPU. This document is pinned to Rust HEAD `0b78102` and C HEAD `2899a74`.

Stance. Every number here was measured from the tree while writing, not copied from a document. Where a project document and the code disagree, the code wins and the disagreement is recorded — there is a list of them in §10, and it is one of the more useful parts of this paper. Roughly a third of the document is about what does not work, is not built, or is not known.

Contents

1	What this document is, and how to attack it	2
2	The idea	2
2.1	The problem	2
2.2	Vocabulary	2
2.3	Mode 1 and Mode 2	2
2.4	The correctness criterion, and why it is not “replay everything”	3
2.5	Why this is hard	3
3	Provenance: the C artifact, and what it actually proved	4
4	The architecture	5
4.1	Layers and ports	5
4.2	The crate dependency graph	7
4.3	The data path	9
4.4	The isolation model	11
4.5	The lifecycle	13
5	The crates	15
5.1	kayfabe-core — 8 411 lines, BUILT and mutation-hardened	15
5.2	kayfabe-abi — 7 455 lines, BUILT — and mislabelled a stub everywhere	15
5.3	kayfabe-linux-raw — 6 097 lines, BUILT — the entire unsafe surface	15
5.4	kayfabe-rt — 2 556 lines, BUILT (L1-M1)	15
5.5	kayfabe-fwd — 2 371 lines, BUILT — and never exercised against a GPU	16
5.6	kayfabe-mocks — 2 156 lines, BUILT , test-only	16
5.7	kayfabe-vmm-kvm — 1 891 lines, BUILT , one open defect	16
5.8	kayfabe-isolate — 1 617 lines, BUILT traits; no real implementation	16
5.9	kayfabe-trace — 1 474 lines, BUILT vocabulary; not threaded	16
5.10	kayfabe-shell — 995 lines, BUILT (L1-M2, in progress)	16
5.11	kayfabe-vmm — 881 lines, BUILT traits only	16
5.12	kayfabe-completion — 756 lines, BUILT	16
5.13	kayfabe-util — 665 lines, BUILT	17
5.14	kayfabe-arch — 556 lines, BUILT traits only	17
5.15	kayfabe-mmio — 309 lines, table BUILT ; walker is 41 lines of nothing	17
5.16	kayfabe-gsp — 34 lines, STUB — and the critical path	17

6	The invariant catalogue	17
6.1	The two-layer trust model	18
7	What is tested, how, and what that does and does not prove	19
7.1	The shape of the suite	19
7.2	The six CI gates	19
7.3	Exercised versus merely present — the project’s own core doctrine	19
7.4	The mean test, and composed versus isolated runs	20
7.5	The conservation ledger	20
7.6	The mutation gate — and why no score should be quoted today	20
7.7	The OS-free configuration, and why it exists	21
7.8	What the suite does not prove	21
8	Discussion: the good, the bad, and the risky	21
8.1	What is genuinely good	21
8.2	kayfabe-gsp is unwritten, and it is the critical path	22
8.3	Nothing has ever touched a real GPU	22
8.4	The quadratic control plane — an open, measured DoS	22
8.5	Reclamation is the least finished plane	23
8.6	arm64 is measured for the core and unmeasured for the shell	23
8.7	The QEMU ≥ 10.2 floor is a distribution constraint	23
8.8	The bench is a single serialised GPU	23
8.9	The recurring failure mode: claims that were true <i>once</i>	23
8.10	Risks this document adds	24
9	How to read the code	25
9.1	Entry points	25
9.2	Conventions	25
9.3	Which design documents to read, in order	25
10	Claims corrected while writing this paper	25
10.1	What this paper could not verify	26
A	Measurement method	27

1 What this document is, and how to attack it

This is a review document. Its job is to describe kayfaber accurately enough that a competent systems engineer who has never seen it can (a) explain what it does and why it is hard, (b) find any component in the tree, (c) judge whether the claims are believable, and (d) find the weak points without help. It is not a status report and it is not a pitch.

Four consequences shape everything below.

Numbers trace to something. Every count, score and measurement names its source. Where a number is an estimate it says so; where a claim could not be verified it is marked **[unverified]** rather than softened. Appendix A gives the exact commands used.

Unbuilt is stated as unbuilt. Most of the planned system does not exist. The build order is deliberate, but a document that blurs “designed” and “running” is worthless for the stated purpose. The layer diagram (Figure 1) and the data-path diagram (Figure 3) both carry explicit **NOT BUILT** bands.

The uncomfortable section is the longest. §8 is where to start if you only read one section.

This document applies the project’s own doctrine to itself. kayfaber’s testing doctrine opens with the rule “a green instrument on an unexercised path is worse than no instrument — it reads as evidence, and nobody re-checks a zero” (docs/design/testing_dochrine.md §1). The same rule applies to a whitepaper. So where this document says a thing is tested, it also says what that test would have to be wrong about for the claim to fail.

The single most important sentence in this paper. No part of the Rust codebase has ever driven a real GPU. There is no binary in the workspace: `find -name main.rs` returns exactly one hit, and it is the offline ABI generator, which lives outside the workspace on purpose. The only composition of vCPU threads, reactor, executor and core that exists is in the test suite (tests/tests/l1_mean.rs:7794--7890). Everything below describes a system that is real, tested and internally coherent, and that has not yet met the thing it exists to talk to.

2 The idea

2.1 The problem

Giving a virtual machine a real GPU normally means one of two things. **IOMMU passthrough** hands the whole physical device to exactly one guest: simple, fast, and it gives every other guest nothing. **Vendor virtualization** (SR-IOV, NVIDIA vGPU) shares the device properly, but it exists on datacenter parts under licences, not on the consumer cards most people own.

kayfaber is a third option: **share one commodity GPU among several untrusted virtual machines, with per-guest-process blast-radius containment, using only unprivileged host processes.** The multi-tenant part is the point. A single cross-tenant leak, or a hostile guest that can wedge the box, would be fatal to the thesis.

2.2 Vocabulary

The rest of this paper leans on five terms.

- **RM** — NVIDIA’s *Resource Manager*, the object model inside the driver. Everything a GPU program touches is an RM object with a handle: clients, devices, address spaces, channels, memory, engine contexts. Guest software allocates and frees them through ioctls and RPCs.
- **GSP** — the *GPU System Processor*: on modern NVIDIA hardware a real CPU core on the GPU die, running a firmware copy of much of the resource manager. The host driver no longer programs most of the hardware directly; it sends RPC messages to the GSP over a ring buffer in memory.
- **VAS / PDB** — a GPU virtual address space, and its *page directory base*: the physical address of the root of that address space’s page tables. The GPU’s MMU keys page tables by PDB, which makes **the PDB the real memory boundary** — not the client handle.
- **Channel / vChid** — channels are the GPU’s work-submission queues; the *virtual channel id* is how a submission is routed to one. **vChid is the execution boundary.**
- **Doorbell** — the MMIO register write that says “channel *N* has work waiting”. The value written is a token that encodes the vChid.

2.3 Mode 1 and Mode 2

The predecessor project defines two modes, and kayfaber is only the second.

Mode 1 forwards the guest's *userspace* NVIDIA ioctls to a host stub that replays them against the real `/dev/nvidia*`. It needs a guest kernel module and a virtio transport, so the guest is modified, and it caps you at "a Linux guest running our module". It is mature: 22 real GPU applications at host parity, Vulkan, OpenGL, a working desktop present path, five security audits.

Mode 2 inverts it. The guest runs the **stock, unmodified NVIDIA kernel driver** against an *emulated* PCI device that presents a plausible GPU and a *faked* GSP. We implement the RPC ring the driver talks to and answer as the firmware would. The guest driver believes it owns hardware: it allocates RM objects, builds GPU page tables, creates channels, and rings doorbells. We are on the other side of that conversation, and from the protocol traffic we reconstruct what the guest program was actually trying to do — then do the equivalent thing on a real host GPU through ordinary unprivileged userspace operations, inside a sandbox. Mode 2 needs *nothing* from the guest, which is the entire reason it is worth the difficulty.

2.4 The correctness criterion, and why it is not "replay everything"

The naive reading of Mode 2 is "intercept the guest's privileged operations and perform them on the host". That is wrong, and the project's forwarding model says so explicitly:

*"The job of Mode-2 is **not** to replay those low-level steps on the host. It is to **recover the original userspace-level intent and re-express it as the normal, unprivileged host userspace operations** — exactly the operations Mode-1 forwards directly."*
[C: docs/design/mode2_forwarding_model.md]

The guest's kernel RM has *already* decomposed an unprivileged userspace intent (`cuCtxCreate`, `cuMemAlloc`, a kernel launch) into privileged GSP-internal steps. Replaying those steps on the host would require privilege we deliberately do not have — and the project has a named lesson about this: an `NV_ERR_INSUFFICIENT_PERMISSIONS` from the host stub *never* means "we lack a privilege we should have", it means "we forwarded at the wrong layer". So the operational split is:

- **Case 1** — the RPC essentially *is* the userspace operation. Re-issue it roughly 1:1 on the host, through the isolate, unprivileged, and let the host's own kernel RM legitimately re-derive the privileged steps for itself.
- **Case 2** — a GSP-internal control with no userspace equivalent (`PROMOTE_CTX`, `GET_CTX_BUFFER_INFO`). **Acknowledge the guest; do nothing on the host.** The host has already done the equivalent for its own context.

The correctness criterion that licenses this is **observable end-state equivalence**, not step fidelity. It is perfectly correct for some guest-kernel operations to complete instantly as fakes, *as long as* the one operation that actually carries the work triggers the real host-side chain. The hard boundary on the other side is equally explicit: a completion that gates guest *userspace* must be a real host write. Forging one would make the whole project a lie, and the design forbids it by name.

2.5 Why this is hard

1. **The protocol is not documented and not stable.** It varies on two independent axes: the *driver version* (wire layouts) and the *GPU generation* (silicon behaviour). kayfabe separates these as "Axis A" and "Axis B" and gives each its own trait seam, because conflating them is its own bug class.
2. **Some things cannot be faked at all.** GR's golden context is produced by FECS/GPCCS microcode on real silicon. There is no way to fabricate one. The design's response is to never emulate an engine — forward the guest's own pushbuffer and let real silicon produce the context.
3. **Address translation is a trust problem, not a lookup problem.** The guest builds GPU page tables in memory it controls. Reading them at an arbitrary instant means acting on bytes an adversary can change under you.
4. **Identity collides by construction.** Guest handles and guest VAs are per-process namespaces. Two processes using identical values is normal. Anything that keys on them is wrong (Figure 4).
5. **The failure modes are global.** A mis-resolved address is not a wrong answer; it is a host GPU MMU fault that takes out a channel, and in the worst case a state the guest driver cannot recover from without a full VM restart.
6. **Everything is concurrent.** *N* guest vCPUs, several guest processes, several host isolate processes, an interrupt path, and a host RM that *serializes every ioctl per client* so that one wedged verb blocks every isolate in D-state.

3 Provenance: the C artifact, and what it actually proved

keyfabe is a rewrite, not a greenfield project. The C artifact at `/workspace/nvidia-gpu-passthrough` is the working prior implementation and the differential oracle for everything the rewrite claims about NVIDIA's behaviour. Being precise about what it did and did not prove is the foundation of every other claim here.

Claim	Mode	What was actually measured
22 real GPU apps at host parity	1	True. <code>tests/perf/realapp_matrix.md</code> , 2026-06-01. Same binary, host and guest, strictly serially on one RTX 3060, with a correctness check alongside throughput; parity gate ≥ 0.90 . 10 CUDA benchmarks, 7 PyTorch, 2 LLM, 3 graphics. Caveat the doc states itself: NVENC is <i>partial</i> , not parity (<code>InitializeEncoder</code> fails); NVDEC excluded (broken on the host baseline too).
7B LLM at 63 tok/s	1	True but mis-framed. 63.4 tok/s is a Mode-1 A/B endpoint after a caching fix, not a headline result. The audited same-day parity figure is host 67.8 $\rightarrow$ guest 66.0 tok/s , $0.97\times$ (Qwen2.5-7B Q4_K_M, <code>-ngl 99</code> , RTX 3060). Quote the ratio.
Mode-2 bare-metal parity	2	True, one workload. 49.9 tok/s Mode-2 vs 47.5 tok/s host-native <i>on the same RTX 3050</i> , i.e. zero forwarding overhead within noise. The entire earlier 20 $\rightarrow$ 50 tok/s gap was nested-virt vmexit tax , not Mode-2 design — the <code>vast.ai</code> bench is itself a VM, so the Mode-2 guest was L2. The project retracted its own earlier “~21% residual” claim as an apples-to-oranges comparison across two different GPUs.
Mode-2 CUDA correctness	2	True. Full fake-boot + GSP RPC + GMMU + CE + IRQ; <code>cuInit</code> , <code>cuCtxCreate</code> , byte-exact <code>matmul</code> ; PyTorch 2.5.1 <i>single-process</i> . Bugs #12 (second-context hang) and #13 (multi-iteration hang) resolved on hardware.
What the C artifact never proved		
Mode-2 multi-process	2	Never. Bug #14 is open. Root-caused on hardware 2026-07-24 by a host <code>Xid 31 FAULT_PDE</code> naming the loser's exact GR channel: the second process's identical guest VAs were never published into <i>its own</i> host address space.
Mode-2 sequential processes	2	Broken at HEAD. Measured 2026-07-25: exactly <i>one</i> CUDA process works per QEMU lifetime; the second fails <code>cuInit</code> $\rightarrow$ 999 regardless of how the first exited. The “fresh boot per clean run” bench discipline is this bug, not a precaution.
Mode-2 graphics / display	2	Never. All Vulkan/OpenGL/present results are Mode 1.
Mode-2 on the 22-app matrix	2	Never run. It is Mode-2's definition of done.
Multi-GPU, MIG, non-Ampere	—	Never. Ampere GA106 only. Turing/Ada/Hopper/Blackwell rows in the ABI doc are all marked <i>assumption</i> — <i>verify</i> .
Mode-2 security posture	2	Not audited. The five security audits are Mode-1-era. The 2026-07-25 bench found live guest-reachable Mode-2 defects, including parsing arbitrary guest RAM as GSP RPC elements after a guest-triggered driver reload.

The structural reason for the rewrite. #14 is not a bug that could be patched. The architecture document states it in one line: “*the emulator's completion, execution, isolate, and GPA-arena planes are each a single-process singleton, and they fail in dependency order.*” Four rounds of investigation each found a different plane; they were one structural fact observed four times. Making per-process separation structural rather than retrofitted is the rewrite's founding requirement, and it is why the diagram in Figure 4 is the centre of the design.

One provenance correction worth carrying. Two C design documents state the C baseline encodes “~14 months of quirk capture”. Git says the first commit is 2026-05-24 and the last is 2026-07-24: **two months, 739 commits**. Either the figure means agent-hours, or it is wrong. Use the verifiable number.

4 The architecture

4.1 Layers and ports

The one defining constraint. The logic core is a pure state machine over guest-supplied bytes: no OS, no syscalls, no hypervisor types, no wall clock, no `#[repr(C)]` NVIDIA struct layouts, and no concrete GPU-generation or driver-version name anywhere. Everything effectful crosses a trait seam. `unsafe_code = "forbid"` is a workspace lint; every core type is compile-time-asserted `Send + Sync`.

This is enforced, not intended. Three CI greps run on every push: a *hexagonal boundary* grep that fails if `eventfd|epoll|timerfd|rawfd|libc|O_NONBLOCK` appears in any of the 11 pure crates; a *VMM-vocabulary* grep that fails if any QEMU concept (`BQL`, `MemoryRegion`, `qemu_bh`, `iothread`, ...) appears in those 11 plus `kayfabe-rt`; and a *GPA-accessor* gate that asserts `.gpa_read(/.gpa_write(` appears **exactly once** in the entire forwarding crate.

The claim that pays for this constraint is the **adapter contract**: bringing up a new GPU generation is `impl Arch` for `<Gen>` in an adapter crate with *zero edits to any logic crate*. The standing proof is that the whole suite runs the real core against `MockArch`, whose encodings are deliberately non-NVIDIA — if the core had absorbed a real encoding anywhere, the mock would not work.

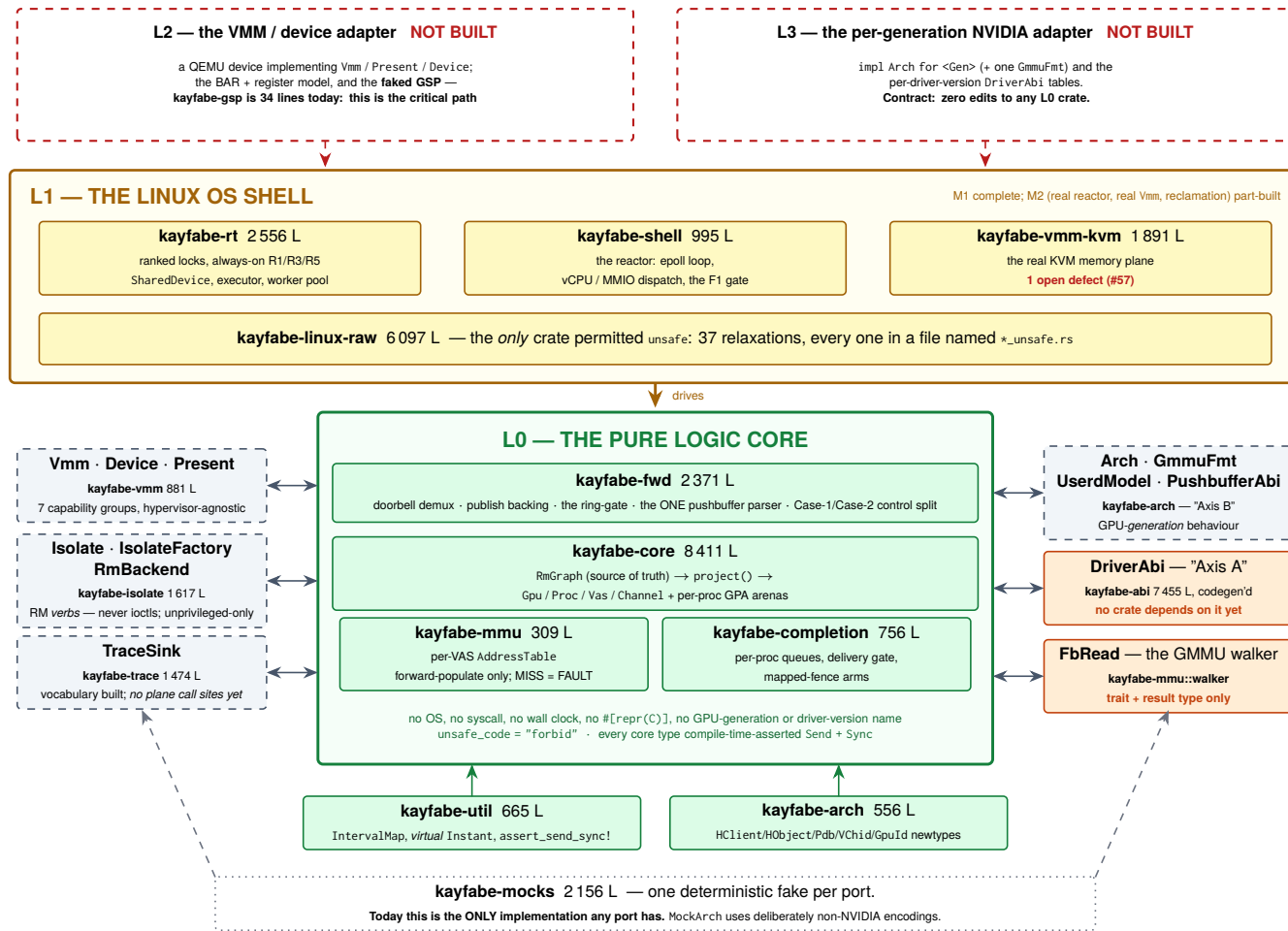


Figure 1: The layer model and the hexagonal ports. Line counts are measured (crates/*/src/**/*rs). Two of the six ports have no real implementation and never have; the two adapter layers that would provide them are unbuilt. kayfaber-mocks is not a convenience — it is the only implementation any port has.

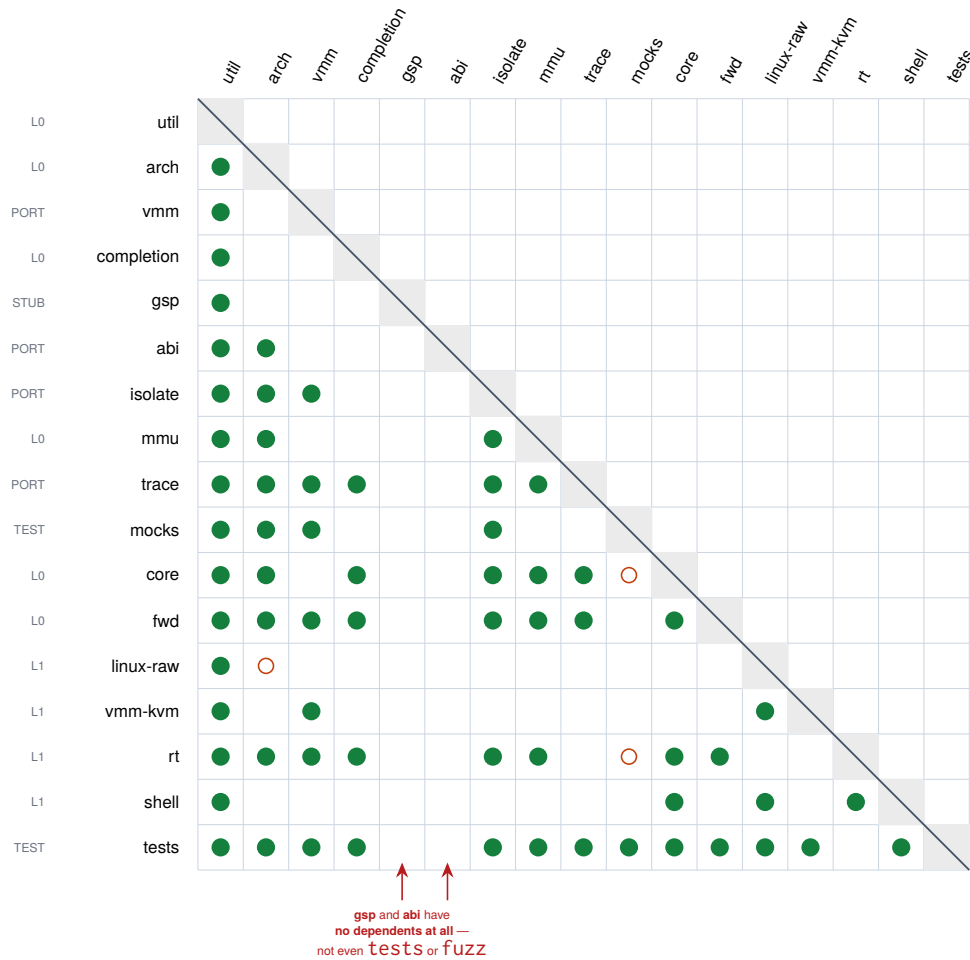
4.2 The crate dependency graph

Figure 2 is derived from every `Cargo.toml` in the tree and cross-checked against `Cargo.lock`; it is not drawn from the design documents. Three findings come straight off it.

The external dependency surface is essentially zero. In the whole workspace there are three external crates: `libc` (in `kayfabe-linux-raw` only), `trybuild` (dev-only) and `proptest` (dev-only). Thirteen of the sixteen crates have no external dependency of any kind. Whatever else is true, the supply-chain surface is not a place to look for problems.

The graph is acyclic, including dev edges, which the strictly lower-triangular shape of the matrix demonstrates directly.

But the claimed strata are not disjoint, and the documents imply they are. `kayfabe-mmio` — a core crate — depends on the `kayfabe-isolate` *port* (it needs `HostHandle`). `kayfabe-core` and `kayfabe-fwd` depend on the `trace` and `vmm` ports. “Core” and “port” are one DAG with a purity rule, not two layers. That is a defensible design (the `trace` crate had to sit *below* `core` so every plane could emit into it), but a reader who takes the hexagon picture literally will be surprised by the manifests.



Reading it. Row r , column c filled $\Rightarrow$ crate r depends on crate c . All 65 [dependencies] edges (●) and all 3 [dev-dependencies] edges (○) are shown; every kayfabe- prefix is dropped.

Strictly lower-triangular in this ordering $\Rightarrow$ the graph is a DAG, dev edges included.

External dependencies, in total: libc (in linux-raw only), trybuild and proptest (dev only). Thirteen of the sixteen crates have *no* external dependency of any kind.

Two things the claimed strata do not survive: mmu (core) depends on the isolate port; core and fwd depend on the trace and vmm ports. "Core" and "port" are not disjoint layers — they are one DAG with a purity rule.

Figure 2: The crate dependency matrix, derived from the manifests. kayfabe-abi and kayfabe-gsp are disconnected islands: nothing in the workspace — not tests, not fuzz — depends on either. For gsp that is expected (it is a 34-line placeholder). For abi it is the more interesting fact: 7 455 lines of working, oracle-tested wire decoding with no consumer.

4.3 The data path

Figure 3 traces one guest GPU operation from a CUDA call to real silicon and back, with the file and line of every step. Three things about it are worth stating in prose.

The plan/execute/commit shape is the whole concurrency design. A host RM verb can block for a long time — the host RM serializes every ioctl per client, and it waits *uninterruptibly*, so one wedged verb parks every isolate that shares the client in D-state. Invariant **R1** therefore forbids issuing a verb while any ranked lock is held. That forces a three-phase shape: *plan* under locks, *execute* with no lock held, *commit* after re-acquiring. And because the locks were dropped in the middle, invariant **R5** requires the commit to re-validate everything the plan assumed. This is not free — §8 covers what it costs.

The ring gate is the #14 fix, and it is structural. Before any host operation, a doorbell submission’s entire working set must already be published into *this* vas’s own host address space. If it is not, the call refuses with NoVas having issued zero host operations. The failure it prevents is exactly the one the C artifact hit on real hardware: the losing process’s identical guest VAs were never published into its own host GR address space, so the host GPU faulted past the shared prefix.

The upper half of the diagram does not exist. There is no register model, no GSP message-queue transport and no RPC decode, so **nothing in the repository produces an `RmEvent` from real guest bytes**. The core’s control-plane input is synthesised by the test suite. This is the single largest gap in the system and it is the critical path (§8.2).

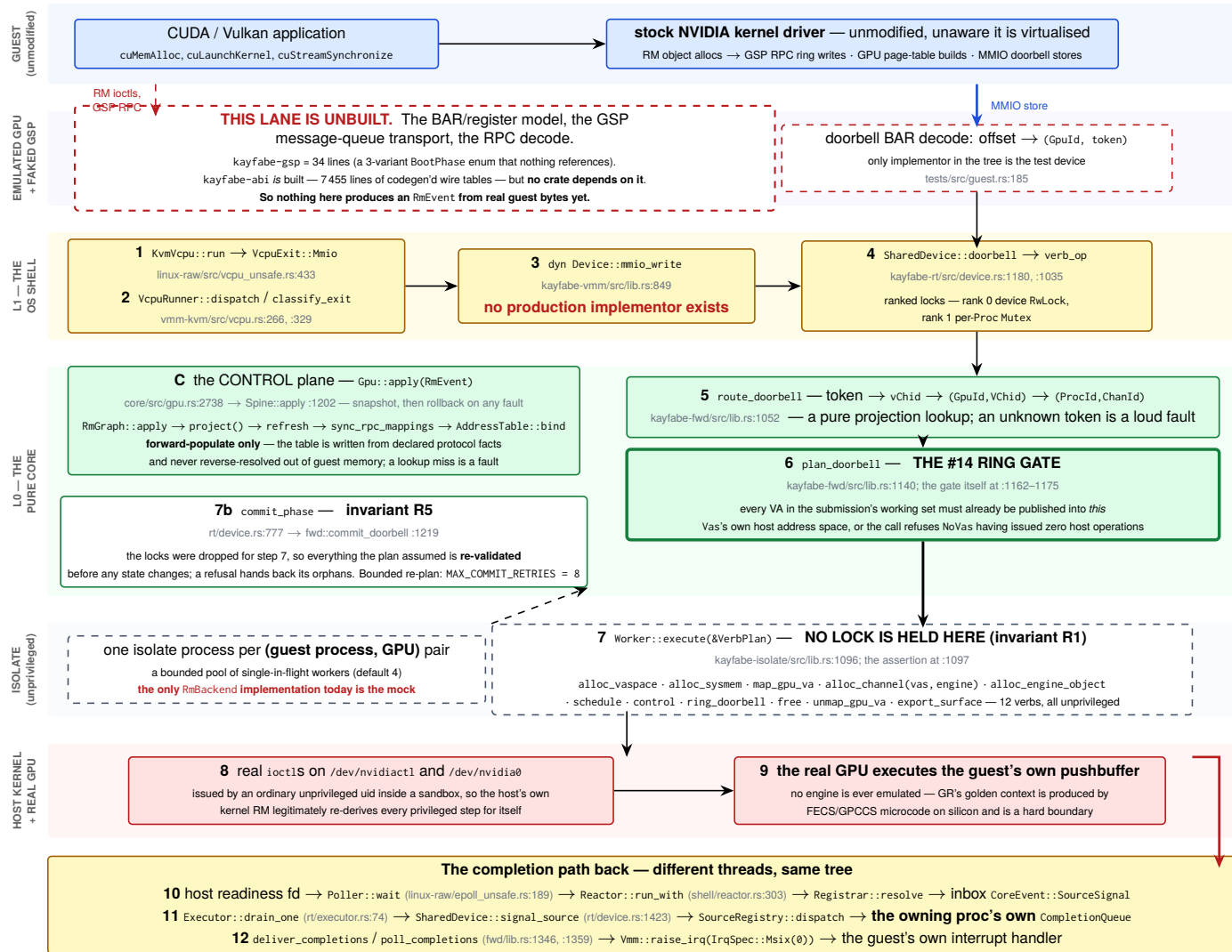


Figure 3: The data path, verified function by function against the tree at 0b78102. Numbered steps are the execution plane; C is the control plane, which populates the address table from declared protocol facts. The red dashed lane is unbuilt.

4.4 The isolation model

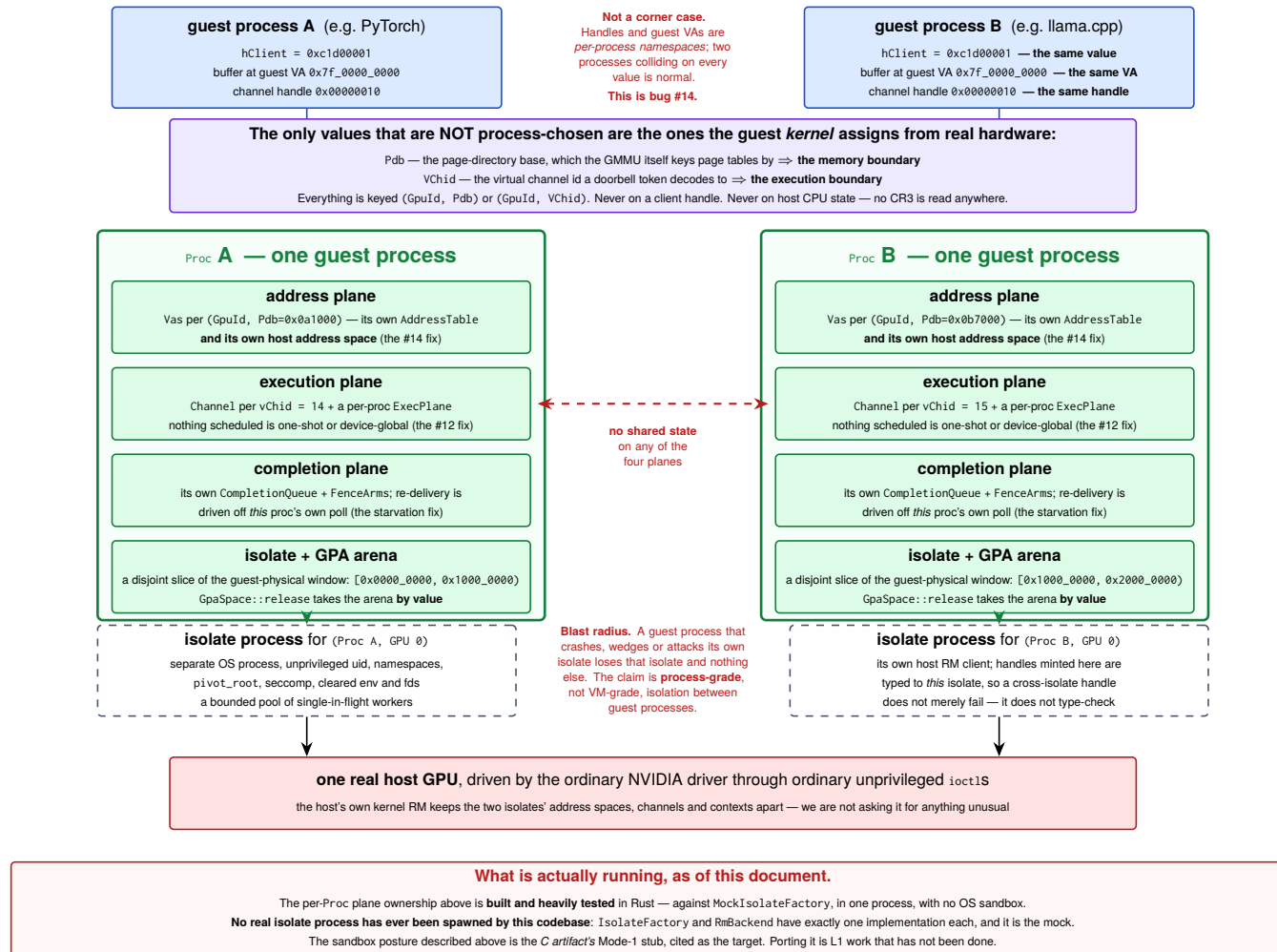


Figure 4: The process and isolation model, and the problem it exists to solve. The per-Proc ownership of all four planes is built and heavily tested against mocks; the sandboxed isolate process at the bottom has never been spawned by this codebase.

The rule that generates the whole model is one sentence: **a guest process is a grouping label, and it is never what an address or execution operation keys on.** Address operations key on (GpuId, Pdb); execution operations key on (GpuId, VChid). Proc exists only to own the isolate, the GPA arena and the lifecycle.

Proc membership is itself a **pure projection** of the RM graph — the dup-connected components of declared user clients — never inferred from timing, never from arrival order. That matters because it means two identical event streams delivered in different orders produce bit-identical process boundaries, which is a property the test suite checks directly by permuting streams.

4.5 The lifecycle

The law every path obeys: RETIRE EAGER, REAP DEFERRED		
Retire is immediate and cheap: the <i>Proc</i> stops accepting operations, its isolates refuse new check-outs. Reap is the heavy half — freeing host objects, killing isolate processes, recycling the GPA arena — and it runs later, at a quiesce point the adapter declares (<code>Gpu::reap_retired</code>). Why: the C artifact proved that reaping at the client-root free <i>hangs the dying context's own residual polls</i>. Reaping never is the #80 leak. Both were paid for on hardware.		
TRIGGER	WHAT HAPPENS	STATUS — read from the tree
T0 a guest frees a subset and keeps running	The most frequent path in a real workload, and the one with no backstop — nothing dies, so nothing reclaims. Dropped host identities move to a <code>pending_release</code> queue on the <i>Proc</i> and drain on a checked-out worker.	Built, and lazier than designed. The drain may fire only when the isolate is otherwise idle (<code>is_quiesced</code>), because no lock can exclude a verb that runs lock-free.
T1 a verb is cancelled mid-flight	The cancel is latched, discharged with no lock held, surfaces as <code>EINTR</code> → Interrupted → <code>FwdFault::Cancelled</code> ; the chain's own unwind frees what it had allocated; the worker is checked back in.	Built and tested (<code>tests/tests/cancellation.rs</code>).
T2 a guest process exits — normally, killed, or killed mid-verb	All three are one path: the guest kernel frees the client root on <code>fd</code> close, so an <code>RmEvent</code> arrives, the projection finds no boundary, and the retire is graph-driven . Every checked-out worker is cancelled; fresh handles become Orphans; the <i>Proc</i> lands on the retired list awaiting T3.	Built and tested.
T3 the GPU goes idle — THE quiesce point	The guest <i>tells</i> us: its driver emits <code>UNLOADING_GUEST_DRIVER</code> (RPC fn 47) on an idle release and re-runs the queue handshake. That re-handshake is the quiesce point, and the C already ran its deferred reap there. Same doctrine as the address table: an observed protocol event, never an inference about idleness.	HALF A LAW. <code>reap_retired</code> exists and <code>CoreEvent::Deferred(DeferredReap)</code> exists — and nothing arms either . Reap-deferred with no trigger is indistinguishable from reap-never in a run that never quiesces.
T4 the guest driver restarts / the GSP re-handshakes	On <code>rmmod</code> , guest panic or VM reset the guest sends <i>no</i> Free events, so the graph, every live <i>Proc</i> , its isolates, arenas, host handles and routing maps all survive into the next driver life — which then derives a second set of components beside the corpses.	NOT BUILT, and the obvious trigger does not exist. Bench-measured 2026-07-26: <code>rmmod</code> emits no fn-47 at all — the idle release at process exit already consumed it. The unload has no observable signal.
T5 an isolate worker dies	Routing is settled (the component is condemned, keyed on its client set). Reclamation is not: a worker that died <i>mid-chain</i> left allocations that are in no Orphans and in no core state. The answer is to escalate to isolate death — <code>SIGKILL</code> — because killing the process is how you reclaim state you cannot enumerate.	Partly built. The design calls this "the single most leak-prone moment"; the carrier type (<code>VerbFailure{err, orphans}</code>) exists.
T6 isolate garbage collection	<code>SIGKILL</code> → the process's <i>exit descriptor</i> becomes readable → reactor → <code>waitpid</code> (non-blocking) → close descriptors, tear down every double-mmap, release arenas, drop worker slots. The kernel already released the host RM objects by closing the connection.	Designed; the OS half is unexercised — no real isolate process has ever been spawned.
T7 whole-device teardown	An explicit ordered <code>shutdown()</code> , never a Drop: stop accepting traps, cancel every worker, wait bounded, retire everything, reap unconditionally, kill every isolate, drain the inbox asserting every remaining event faults rather than mutates, join the threads with no lock held, and assert the conservation ledger balances . Drop is a tripwire that logs loudly if <code>shutdown()</code> was skipped.	Designed; partly built.
The instrument all eight are measured by — the conservation ledger <code>HostLedger, kayfabe-mocks/src/lib.rs:1241</code> Replays the recorded verb log and demands: every acquisition matched by exactly one release, nothing released that was never acquired — objects <i>and</i> mappings. A test may declare a <code>ResidueClaim</code> for what it knowingly leaves outstanding, and the counts are compared for equality, not as a bound: less residue than declared fails too, so a fix that closes a leak must delete the claim that excused it. Its first census over the mean run reported zero leaks — and that was a true negative, not a pass. The workload it was measuring only ever added and removed <i>RPC</i> bindings, which own nothing host-side. Once a phase was written that actually allocates before it frees, the honest baseline was 24 objects, 6 mappings and 24 576 GPA bytes leaking, linear in frees.		

Figure 5: The eight teardown triggers and the retire/reap split. Two of the eight are honestly broken and say so: T3’s quiesce trigger is designed but nothing arms it, and T4’s obvious trigger was measured on hardware and does not fire.

The lifecycle is where the C artifact's scar tissue is thickest, and it is worth understanding why the split exists. Reaping resources at the moment the guest frees its client root *hangs the dying context's own residual polls* — bench-proven. Not reaping at all exhausts the shared guest-physical window after a handful of process generations — also bench-proven, as bug #80, and *re-introduced and re-found in the Rust core* by the regression test written for it. Hence: retire eagerly (refuse new operations, cheap), reap later at a quiesce point the adapter declares.

Figure 5 marks two of the eight triggers red, and the reasons are different in an instructive way. T3 is a design that is not wired: `reap_retired` exists, `CoreEvent::Deferred(DeferredReap)` exists, and nothing arms either. T4 is worse — the trigger the design chose *was measured on hardware and does not fire*. The design assumed `rmmod` emits RPC function 47; the bench showed it emits none, because the idle release at process exit already consumed it. The problem is not that two triggers are indistinguishable; it is that the driver unload has *no* observable signal.

5 The crates

Sizes are `crates/<name>/src/**/*.rs`, measured at 0b78102. "Maturity" is this document's reading of the tree, cross-checked against `docs/design/crate_maturity_map.md`.

5.1 kayfabe-core — 8411 lines, **BUILT** and mutation-hardened

Owns the spine. `RmGraph` is the source of truth: a refcounted resource/handle split with `DUP_OBJECT` aliasing, order-tolerant "parked" facts (a mapping that arrives before its target is held, not rejected), mapping refcounts and capacity bounds. `project()` is a pure function from the graph to process boundaries and the two routing tables. `Gpu::apply` is transactional — snapshot, mutate, re-project, roll back on any derivation fault, so a hostile event earns only its own refusal. `gpa.rs` owns the guest-physical window and carves per-process arenas; `GpaSpace::release` takes the arena *by value*, which makes releasing a live process's arena unrepresentable rather than merely wrong.

Deliberately not here: anything about wire formats, any NVIDIA constant, any OS call.

5.2 kayfabe-abi — 7455 lines, **BUILT** — and mislabelled a stub everywhere

Axis A: the per-driver-version wire layouts. Contains an offline generator (`crates/kayfabe-abi/gen`, a separate cargo project with zero third-party dependencies) that parses NVIDIA's own FINN-emitted C headers from the open kernel modules and emits Rust; 11 generated structs plus one hand-transcribed variant; a version-dispatch table keyed on full `major.minor.patch`; and 2272 lines of tests. The `#[repr(C)]` mirrors are a *layout oracle only* — decode is `from_le_bytes` out of a byte slice, never a cast, so `forbid(unsafe_code)` still holds.

Its test design is the best in the repository: layouts are pinned against **three independent oracles** — NVIDIA's own headers, `gVisor`'s independent transcription, and the C artifact — with a `file:line` citation per number, and a test whose name is `nvos46_the_three_oracles_disagree_and_here_is_exactly_how`. That test found that the C artifact's *parity test* was weaker than the C code it was supposed to guard.

Two honest facts. No crate in the workspace depends on it. And `README.md`, `ARCHITECTURE.md` and its own `Cargo.toml` description all still call it a stub.

5.3 kayfabe-linux-raw — 6097 lines, **BUILT** — the entire unsafe surface

The only crate permitted unsafe, and the only one that does not inherit the workspace lints (because `forbid` cannot be relaxed by an inner `allow`). It restates `unsafe_code = "allow"` plus `clippy::undocumented_unsafe_blocks = "deny"`. Every file that uses the keyword is named `*_unsafe.rs`, so `ls` enumerates the audit surface: 7 files, **37 relaxations**, held by a CI ratchet (`EXPECTED_UNSAFE_BLOCKS=37`) that fails on any change in either direction. It also carries a `trybuild` compile-fail matrix — 9 files that must *not* compile — covering things like a bare `MAP_FIXED` address, a page-size literal, and a view outliving its region.

5.4 kayfabe-rt — 2556 lines, **BUILT** (L1-M1)

The threaded shell. `LockRank{Device=0, Proc=1, Leaf=2}` with an always-on (not `debug_assert`) rank check that panics *before* the OS acquire, so an inversion is deterministic. `SharedDevice` is the plan/execute/commit driver, and it ships **both** lock modes — `Sharded` and `Degenerate` — from day one, tested as first-class configurations with a differential asserting a bit-identical end state, specifically so a late granularity change is never the untested mode.

5.5 kayfabe-fwd — 2371 lines, **BUILT** — and never exercised against a GPU

The entry points an adapter calls: `handle_doorbell`, `publish_backing`, `parse_pushbuffer` (the one parser: CE page-table-write capture, semaphore release, TLB invalidate, everything else opaque and passed through), `forward_engine_object` and `route_control` (the Case-1/Case-2 split), the fence arms, and the present route. This is the crate whose correctness is least established, because its correctness is a claim about NVIDIA's behaviour and nothing here has met NVIDIA.

5.6 kayfabe-mocks — 2156 lines, **BUILT**, test-only

One deterministic fake per port, plus the shared verb recorder and the conservation ledger. Excluded from the mutation denominator on the argument that mutating an instrument is a category error — a surviving mock mutant is not a coverage hole. *Strictly*, it is not confined to dev-dependencies: `tests/` and `fuzz/` list it as a normal dependency, so `cargo build --workspace` compiles it. Nothing shipping links it.

5.7 kayfabe-vmm-kvm — 1891 lines, **BUILT**, one open defect

The real KVM memory plane: real `/dev/kvm` file descriptors, real memslots, real vCPU run loops, MMIO exit classification. **Defect #57: the R1 assertion trips roughly 1 in 6 runs under race.** This is why the QEMU adapter has not been started — building a hypervisor adapter on a memory plane with a known lock violation under contention means any later hang has two candidate causes instead of one.

5.8 kayfabe-isolate — 1617 lines, **BUILT** traits; no real implementation

The port for the unprivileged host worker: `RmBackend` (12 verbs, all of which must be issuable unprivileged), `Isolate` (two-stage retire, cancellation, quiesce predicate), `IsolateFactory`. `Worker::execute` is the single door to a backend and asserts R1 on entry. A `HostHandle` carries its `IsolateId`, so a cross-isolate handle does not merely fail — it does not type-check.

5.9 kayfabe-trace — 1474 lines, **BUILT** vocabulary; not threaded

The typed `TraceEvent` vocabulary, a `TraceSink` port, one-counter total ordering, perf-budget counters and a projection differential. It exists because the GSP milestone's oracle *is* trace replay, so the replay format had to be defined before the harness that consumes it. Two findings from building it are worth carrying: it had to sit *below* `kayfabe-core` (every plane must be able to emit), so it cannot name `RmEvent` or `ProcId` — the bridge is a trait the owning crate implements with an exhaustive match, so a new variant upstream fails the build until the vocabulary names it. And no `&mut Trace` is threaded through any plane signature yet; the vocabulary is driven only from the conformance suite's seam observer.

5.10 kayfabe-shell — 995 lines, **BUILT** (L1-M2, in progress)

The reactor: an epoll loop, source resolution, the inbox, and the executor's wake. It carries the restated F1 rule — *assert a bound, never an absence* — as a concrete quantity: after 16 consecutive unproductive wakes it refuses with `ReactorFault::UndrainableSource` rather than spinning. That restatement came from the mutation gate: three spin-versus-park mutants survived precisely because "never busy-poll" is an absence, and an absence is not observable.

5.11 kayfabe-vmm — 881 lines, **BUILT** traits only

The hypervisor and display ports: `Vmm` (7 capability groups), `Device`, `Present`. An eighth group — the memory-lock primitive — was *removed* from the trait when it turned out to need no hypervisor cooperation on any backend, which is a good sign about the seam's discipline. **Nothing implements `Device` in production code;** the only two implementors in the tree are test types.

5.12 kayfabe-completion — 756 lines, **BUILT**

Per-process queues (pending → in-flight → awaiting-ack → ack), a drain-gated delivery plane whose re-post is driven off *the owner's own poll* (the starvation fix), and mapped-fence arms with the #12 backwards-jump guard. Scored 100% on the mutation gate.

5.13 kayfabe-util — 665 lines, **BUILT**

IntervalMap (the address table’s container, loud on overlap, empty and wrap), a *virtual* Instant so no wall clock exists anywhere in the core, the assert_send_sync! compile-time gate, and the lock witness that invariant R1 asserts against. The witness lives here rather than in kayfabe-rt specifically so that kayfabe-isolate, which cannot depend on rt, can still assert it.

5.14 kayfabe-arch — 556 lines, **BUILT** traits only

The identity vocabulary (HClient, HObject, Pdb, VChid, ClassId, GpuVa, Gpa, GpuId, EngineKind) and the Axis-B port set. Zero real implementations; MockArch is the standing proof that a real one will need no core edits.

5.15 kayfabe-mmio — 309 lines, table **BUILT**; walker is 41 lines of nothing

AddressTable is complete and is the enforcement point for the project’s most-cited directive: forward-populate only, unbind eagerly, and **a miss is a fault** — no reverse resolve, no heuristic pick, no MRU fallback exists anywhere in the codebase.

The walker module is 41 lines containing one trait and one enum, **with no walk function of any kind**. WalkResult is constructed nowhere; FbRead is implemented nowhere. The module’s own doc comment says the walk algorithm is “also here”. It is not.

5.16 kayfabe-gsp — 34 lines, **STUB** — and the critical path

A three-variant BootPhase enum, referenced by nothing. Section 8.2 is about this crate.

tests/ — the conformance suite

39 469 lines of test code against 38 224 lines of implementation. 28 files in tests/tests/, 5 more under crates/*/tests/, and a Scenario DSL in tests/src/. The suite is a workspace member with a [lib], which is why kayfabe-mocks compiles as a normal dependency.

6 The invariant catalogue

These are the properties the design is willing to be judged on. Where an invariant has a mechanism, the mechanism is named; where it is only prose, that is stated, because the project’s own doctrine is “*prefer a mechanism to a sentence — a rule stated only in prose has no reader.*”

Invariant	Statement	Enforced by
Order-independence	Everything derived is a pure function of declared protocol facts, never of arrival order. “Protocol, not trace.”	Permutation/interleave/dup proptests over the whole observable end state (tests/tests/determinism.rs).
MISS = FAULT	Tables are forward-populated only; a lookup that misses is a loud fault, never a guess. Refined: a ~28-site audit found the absolute had a second, correct answer — the real rule is decided <i>per site</i> : <i>not yet knowable</i> ⇒ DEFER; <i>never knowable</i> ⇒ FAULT. A mapping arriving before its PDB bind defers; a handle resolving to the wrong <i>kind</i> faults. Getting it wrong is asymmetric: fault-when-it-should-defer hangs the guest, defer-when-it-should-fault is a security question.	AddressTable has no reverse-resolve path to call. tests/tests/miss_taxonomy.rs makes the taxonomy executable. <i>No shipped site needed a behaviour change; three doc claims did, two of them stating the literal opposite of their own code.</i>
Per-target keying	Pdb and VChid are per-GPU namespaces. Cross-GPU identical ids are legal; same-target duplicates are loud collisions.	project(); tests/tests/multi_gpu.rs.
Per-Proc isolation (I1)	Identical guest VAs and handles in two processes reach disjoint arenas, disjoint host address spaces and disjoint isolates <i>by construction</i> .	Types (GpaSpace::release by value; HostHandle carries its IsolateId) plus a proptest against an injective oracle.
One gated ring path	A doorbell can only reach the host after the #14 working-set gate.	See the correction in §10 — the property holds, the cardinality claim in four documents does not.

Invariant	Statement	Enforced by
Completion integrity (I2)	A completion fires only from a genuinely-armed source in the <i>owning</i> process, at most once; the system forge path is typed so it cannot reach a user process's queue.	Type-level (<code>Traffic::System</code>) plus a differential against an independent reference fence model.
RefCount soundness (I3)	A resource is alive $\iff$ it has a live handle or map reference. No premature destroy, no leak, dup survives origin free.	Proptest over deep trees with cross-client dups and interior frees.
DoS containment (I4)	<code>Gpu::apply</code> is transactional; every guest-growable table is capacity-bounded with a loud refusal, never OOM.	Corrected, and the correction is in the design doc: "no $O(n^2)$ path on hostile floods" is false and was measured . The caps bound <i>memory</i> , not <i>time</i> — see §8.
R1	No blocking host verb may be issued while any ranked lock is held.	<code>assert_lock_free</code> at <code>Worker::execute</code> and <code>Worker::with_rm</code> — the only two doors to a backend. <i>It does not guard</i> Drop, which is a real call site, and one R1 violation found this week was exactly an Arc ownership issue in a Drop.
R3	Locks are acquired in strictly increasing rank, at most one per rank.	<code>check_acquire</code> , always-on, panics before the OS acquire.
R5	After re-acquiring, re-validate everything the plan assumed.	The four <code>commit_*</code> functions; bounded re-plan at <code>MAX_COMMIT_RETRIES = 8</code> .
F1	Three unrelated things are called F1 in this project: a fixed threat-model bug, the per-GPU collision guard, and the reactor's bounded-poll rule.	All three real; the overloading is a documentation hazard, not a code one.
GL1–GL13	The guest-memory-lock invariants.	Documentation only. Zero occurrences in any .rs file. They have no port to be enforced at, because capability group 8 was removed from the Vmm trait. Treat as a designed, unimplemented subsystem.

6.1 The two-layer trust model

One normative rule deserves its own space, because it decides designs and because it is the cleanest statement the project has produced:

Layer 1 (trap + lock) gives SECURITY against a guest that ignores the protocol. Layer 2 (NVIDIA's own synchronisation points) gives CORRECTNESS for a guest that follows it. Shared tables are NEVER authoritative for an immediate decision.

[docs/design/core_security_threat_model.md §2.1]

Neither layer is a weaker version of the other. Layer 2 alone is not security: a hostile guest can update a shared table *after* validation and *before* we act, and it can fake the handshake entirely. Layer 1 alone is not correctness: a trap says the bytes cannot change while you look; it does not say they are *finished*. A cooperating guest halfway through publishing a page-directory update has written something internally inconsistent and perfectly stable, and reading it under a perfect lock yields a perfectly-locked wrong answer.

The consequence — *a shared table is evidence, never authority* — is why the address table is forward-populated and why a miss is a fault.

And the C's measurement sharpened it in a way worth carrying. The Layer-2 edge is *whichever* edge the protocol commits at, not a favourite one. On the kernel/UVM/RM paths that edge is the TLB invalidate.

On the GSP-emulated compute path it is *not*: round 6 of the #14 investigation measured **zero** occurrences of both invalidate transports there. A design that had encoded “read at the invalidate” as *the* Layer-2 rule would have had no synchronisation point at all on the path that matters most. The general rule survived; one of its instances did not.

What the C artifact does not have is Layer 1. There is no lock of any kind between QEMU and an isolate in a system that runs 22 real GPU applications at host parity. It ships a copy-once snapshot instead, and the reasoning is written out in its source. The working system to date has been running on Layer 2 alone — which is correct for a cooperating guest and is not a security property.

7 What is tested, how, and what that does and does not prove

7.1 The shape of the suite

Measurement	Value	How
Tests reported green	515	KAYFABE_NO_KVM=1 KAYFABE_SLOW=1 cargo test --workspace, recorded in commit 853378
#[test] attribute sites	512	grep over workspace members (excludes the detached generator)
proptest! blocks	16	each is one test that runs many cases
Checked-in proptest failures	0	find for proptest-regressions: nothing exists
KVM-gated tests	36	CI floor asserts ≥ 35 were <i>reached</i>
TSan targets	4 files, 65 tests	nightly only
Fuzz targets	1	parse_pushbuffer; CI compiles it, does not run it
Implementation lines	38 224	crates/*/src/**/*rs
Test lines	39 469	tests/, crates/*/tests/, fuzz/

The suite runs in about 20 seconds with no GPU, no OS dependency and no wall clock. That is a deliberate and load-bearing property, not a convenience: it is what makes the adversarial suites affordable enough to run on every change.

7.2 The six CI gates

There is one workflow file with six jobs. Note the README says “four standing CI gates”; that is stale (§10).

Job	When	What it asserts
stable	every push + PR	Eleven steps: build; test with KVM forced absent ; assert the KVM-gated tests were <i>reached</i> ; clippy -D warnings; fmt --check; the hexagonal-boundary grep; the VMM-vocabulary grep; the unsafe-surface 1s gate; the GPA-accessor gate (exactly one call site); three unsafe-containment asserts including the ratchet at 37; and fmt --check for the fuzz workspace.
aarch64	every push + PR	cargo check --workspace --target aarch64-unknown-linux-gnu
nightly-fuzz	every push + PR	the fuzz harness <i>compiles</i>
slow	nightly	the full suite with KAYFABE_SLOW=1
tsan	nightly	ThreadSanitizer over four threaded targets, -Zbuild-std so std’s own synchronisation is instrumented
mutants	weekly	cargo mutants over every production crate

7.3 Exercised versus merely present — the project’s own core doctrine

This distinction is the most useful thing in the repository, and it is worth reproducing the incident that generated it, because it is the template for every criticism in §8.

The conservation ledger’s first census over the mean run reported **0 leaked objects** — and that was a **true negative, not a pass**. A design document claimed the workload “exercises T0 thousands of times per run”; the churn it actually drives adds and removes *RPC* bindings, which own nothing host-side, and the script’s two real subset-frees targeted channels that phase 0 deliberately leaves virgin. **The path had never been reached.** Reaching it took a new test phase. Once reached, the honest baseline was **24 objects, 6 mappings and 24 576 GPA bytes leaking, linear in frees**.
[docs/design/testing_doctrine.md §1]

Three rules follow, and the project applies them: a reclamation test that does not first allocate proves nothing; every instrument needs a non-vacuity assertion (a case where it is known to read non-zero); and **bite-check the instrument, not only the fix** — reverting the fix must move the number, and if it does not, the number is not measuring the fix.

Bite-checking is real practice rather than aspiration, and the evidence is that the repository records bite-checks that *did not bite*: at least six such notes are written into the source at the site they concern, each explaining what the neutering failed to move and what was added as a result. One has been promoted into a permanent test that plants a gap and a repeat in a trace and requires the order checker to fire. A commit message from this week reads, in part, *"the bite harness itself lied twice."*

The doctrine also produces two mechanisms directly. Slow tests are gated by `KAYFABE_SLOW`, and when unset they print `SKIPPED (slow): ... straight to stderr`, bypassing `libtest`'s capture; nothing in the suite is `#[ignore]`d, on the argument that `#[ignore]` is invisible in a summary and impossible to count. And the KVM gate prints on *both* arms — `KVM-GATE: RAN` or `KVM-GATE: SKIPPED` — with CI asserting a floor on the sum, because the reverse failure (all 36 gated tests silently vanishing while CI stays green) is equally silent.

7.4 The mean test, and composed versus isolated runs

`tests/tests/l1_mean.rs` is the largest single test file (35 tests) and is designated *the arbiter*. Its standing is: *pass = the design survived contact; fail = the doc changes, not the assert*. The rule it exists to enforce is an owner directive with four obligations — drive realistic multi-process $\times$ multi-thread $\times$ multi-GPU traffic end to end through the mocks and assert the observable end state; make it mean rather than happy-path (free mid-flight, race a re-declaration against a still-publishing generation, kill a worker partway, run both lock modes); wire every new milestone into it; and **never narrow a test to make it pass**.

The justification is measured, not asserted. One identity round landed 7 bite-checks; **none of the 363 pre-existing tests caught any of them**. Every real defect that day came from a composed run or a newly sharpened criterion. In the doctrine's words: *"the happy path found nothing, all day."*

A representative finding: a test named `worker_death_retires_the_proc_loudly_and_never_resurrects` was green — and green *only because it never issued an apply after the HUP*. The composed run put a worker hangup and an alloc/map-heavy workload in the same run and found that an out-of-band retire is undone by the next refresh: a guest able to crash its isolate worker got a **clean new isolate** on its next RM event. The first fix for the related leak class was a use-after-free of its own making.

7.5 The conservation ledger

The instrument all eight teardown paths are measured by. `HostLedger` replays the recorded verb log and demands every acquisition matched by exactly one release and nothing released that was never acquired — objects *and* mappings. A test may declare a `ResidueClaim` for what it knowingly leaves outstanding, and the counts are compared **for equality, not as a bound**: *less* residue than declared also fails, so a fix that closes a leak must delete the claim that excused it. There is deliberately no `allow_leaks` escape hatch.

Its stated limit, recorded in the source: the ledger increments only on *success*, so it says nothing about failed verbs. That is why a separate `VerbFailure{err, orphans}` type exists to carry handles out of a chain that died partway.

7.6 The mutation gate — and why no score should be quoted today

The gate mutates the code and measures whether the suite notices. It has twice been the most valuable instrument in the project and twice been silently wrong *upward*, which is the failure mode the project cares most about.

The first lie: it tested nothing. `kayfabe-fwd` and `kayfabe-rt` have no unit tests of their own; without `--test-workspace true`, cargo-mutants ran an empty suite and reported everything missed.

The second lie: compiler crashes vanished from the denominator. cargo-mutants rewrites one file per mutant in a copied tree that reuses one incremental dep-graph cache, and that cache does not survive the churn — rustc ICEs. cargo-mutants cannot tell a compiler crash from a type error, so it files the mutant *unviable* and **silently removes it from the denominator**. Measured: **136 of 293 mutant builds ICE'd**, turning a real 88.95% into a reported 67%. CI now fails on any ICE in a mutant log, and on a zero viable count, *before* the threshold is read.

The historical numbers, each true of the scope it was measured on: **99.2%** (245/247) on the pure L0 core, with both residuals argued equivalent or acceptable; **92.44%** (159/172) on the L1 threaded surface, where the residual untested *logic* is 2 mutants wide and named.

Do not quote a current mutation score. On 2026-07-27 the scope changed from four hand-picked paths to every production crate — the old scope covered roughly 41% of the implementation, and *every real defect that week lived in the unmeasured part*. The CI threshold of 91% was derived against the old scope, so it now describes a different population. The workflow marks it **PENDING RE-DERIVATION** in its own text. A full-scope run has been measured once at **90.60%** raw, rising to roughly **94.9%** once test scaffolding is excluded from the denominator (48% of that run's misses were mutations of mocks), but neither number has been confirmed by a second run.

7.7 The OS-free configuration, and why it exists

KAYFABE_NO_KVM=1 forces the KVM probe to report the device absent, so the pure logic core plus mocks — no OS layer at all — runs on purpose on any box. CI runs *this* as its default test configuration.

The reason it was made explicit is the doctrine applied to itself. GitHub runners have no `/dev/kvm`, so CI was already running the OS-free configuration — **by luck of what the runner image happens to lack**. It was unreproducible on any developer box, and it would have lapsed silently the day a runner image started shipping the device. In the commit's own words: *"a guarantee that holds by luck is the same shape as a green instrument on an unexercised path."*

The owner's question that prompted it is worth repeating because it is the right question: *"I hope you didn't delete the mocks so the core can still be tested without OS layer? and that must pass as well so we don't lock in architectures/os semantics/qemu and later regret."*

7.8 What the suite does not prove

- **It does not prove anything about NVIDIA.** Every test runs against MockArch, whose encodings are deliberately *not* NVIDIA's. That is the right design for proving the seam, and it means the suite cannot catch a wrong belief about the real protocol. Only the C artifact and the bench can do that.
- **The with-KVM configuration has no automated gate.** CI runs *only* the KVM-absent configuration. The 36 tests that use a real `/dev/kvm` — including the entire real memory plane and the vCPU/MMIO dispatch — run on developer boxes and the bench, and nowhere else. This is the largest hole in the CI story, and the workflow says so in its own text rather than hiding it.
- **The fuzzer is compiled, not run, by CI.** One target, over the one place raw guest bytes reach the core. Its two historical findings were minimised into deterministic in-tree regressions, which is the stronger form. The corpus is git-ignored, so a fresh clone starts from empty.
- **Single-threaded logic testing of a concurrent system.** The core is tested deterministically because it is a pure state machine; the concurrency is tested separately by the threaded suites plus TSan. The argument is sound, but it depends on the core genuinely having no interior mutability — which is asserted at compile time, and is the sort of property worth re-checking if you are looking for holes.
- **The page-size axis is designed and not built.** The testing doctrine cites [4 KiB, 16 KiB, 64 KiB] as an application of its "both modes ship from day one" rule. The environment variable that would select it does not exist in any `.rs` file — only in forward-looking comments. The `LockMode` half of that same rule *is* built and exercised.

8 Discussion: the good, the bad, and the risky

8.1 What is genuinely good

The purity constraint is real and it is mechanised. A "pure core" claim is usually aspirational. Here it is three CI greps, a manifest-level `forbid`, a naming rule that makes `1s` the audit tool, and a ratchet on the exact number of `unsafe` relaxations. The external dependency surface is three crates, two of them dev-only.

The failure ledger is unusually honest and it is load-bearing. Two design documents carry "contact logs" — `11_concurrency.md` §12 and `11_os_shell.md` §14 — recording every place building the code proved the design wrong. They are long. They contain entries whose titles are things like "the criterion was wrong", "boundary-1 broken", "two sections of §3 contradict each other", "the design was wrong about its own severest claim". A project that writes these down is a project whose other claims are worth more.

The adversarial suites found real bugs before hardware existed. 15 at L0, more since at L1, including two control-plane *wedge* bugs reachable by a hostile guest process — a parked `SET_PAGE_DIRECTORY` and a parked `MAP_MEMORY_DMA`, each of which could be aimed via a `dup` alias to make a benign third party's allocations fail forever. Both were found by a property test, not by inspection.

Both lock modes and both fallbacks ship as first-class tested modes. The argument is sharp: the slow path is the path the system takes *when something has already gone wrong*, and a fallback exercised only by the fast

path declining has test coverage proportional to how often the fast path fails — which is the quantity the optimisation exists to drive to zero.

The ABI oracle design. Pinning generated layouts against three independent readings, with a test that documents exactly how they disagree, is better than what most projects do, and it caught the C artifact’s own parity test being weaker than the code it guarded.

8.2 kayfabe-gsp is unwritten, and it is the critical path

34 lines. A `BootPhase` enum with three variants, referenced by nothing, in a crate that depends only on `kayfabe-util`.

Everything the guest driver believes about its GPU comes through this crate. Until it exists there is no register model, no message-queue transport, no RPC decode — and therefore **nothing that turns real guest bytes into the `RmEvents` the core consumes**. The core is a very well-tested function with no production caller on its main input.

The port plan for it is written (1 422 lines) and it is careful about what it does and does not know. Two facts from it should worry a reviewer:

1. **No available source tree is the bench’s driver.** The specification (NVIDIA’s open kernel modules) is version 610.43.02. The independent corroboration (`nouveau`) carries r535 and r570. The bench runs 580.159.04, which sits between r570 and 610 with no vendored source. **Three of the boot path’s structures have three different shapes across the three available trees.** The plan’s own §9 lists nine specific disagreements.
2. **The plan found two of its own briefed premises to be false while being written**, and says so in the commit message. One of those was a claim about the C’s behaviour cited from a C source comment. The doctrine drawn from it: *citing a comment is still citing a belief*.

The mitigation is that its oracle is trace replay against the C emulator’s recorded boots, and the replay vocabulary was deliberately built *first* so the harness would not hard-code the format in the wrong crate. That sequencing decision is good. It does not reduce the size of the gap.

8.3 Nothing has ever touched a real GPU

Stated once more because it is the thing most likely to be forgotten between here and the end of the paper. There is no binary. `Device` has no production implementor. `RmBackend` and `IsolateFactory` have exactly one implementation each and it is the mock. No isolate process has ever been spawned. The composition of threads that would constitute a running system exists only inside one test file.

What this specifically means for the claims in this document: everything about *structure* (the DAG, the purity, the type-level isolation, the lock discipline) is verifiable today and this paper verified it. Everything about *behaviour against NVIDIA* is inherited from the C artifact and re-expressed, and the re-expression has not been checked against the thing it models. The gap between “our mock says the core does the right thing” and “a real driver agrees” is exactly the gap the C artifact spent two months and 739 commits closing, once, in C.

8.4 The quadratic control plane — an open, measured DoS

The invariant catalogue’s I4 says hostile input is *contained*: no wedge, no OOM, no bystander corruption, every hostile event earning only its own refusal. All of that is property-proven. It says nothing about *cost*, and the project measured the cost and recorded the result against itself:

The control plane is $O(\text{live objects})$ per event, so N events cost $O(N^2)$. Measured on a debug build: **1 000 events 0.85 s; 8 000 events 54.8 s**. The capacity cap is 2^{18} live handles, so a guest can drive that curve deliberately — and **PyTorch startup reaches it benignly**. Deleting the per-event graph clone saves ~24% and leaves the curve quadratic, so the clone is neither the cause nor the fix.

The stated statement in `ARCHITECTURE.md` — “no $O(n^2)$ path on hostile floods” — is marked **false** in the same document. Two candidate fixes are named and neither is small: incremental derivation, which is a redesign of the “derived state is a pure function of the graph, never accreted” decision that the entire determinism differential rests on; or an undo journal in the most safety-critical file in the repository. A sibling of the same species *was* fixed (a condemned-list carry-forward, $O(n^2 \log n)$, 55 s $\rightarrow$ 3.8 s, by union-find), which is evidence the class is tractable and also evidence that it recurs.

8.5 Reclamation is the least finished plane

Figure 5 marks two triggers red. Restating the sharper form:

“Retire eager, reap deferred” is currently half a law. The deferral has no bound and nothing arms it. In a run that never quiesces, reap-deferred and reap-never are indistinguishable.

The one genuine cross-process reference has a disposition that no ledger will like. On a kernel/UVM dup of a user address space, refcount zero and per-object Free are not the same event: the last reference retires and reaps the owner *without issuing a single Free verb*, and the disposition of record is the isolate process’s death. GPA is conserved independently and asserted. But any future reclamation ledger, quota or accounting hook will be wrong on that path first — which the design says, in advance, in its own words.

And the most leak-prone moment has no vocabulary at all until L1-M2 lands. A worker that dies *mid-chain* cannot run its own unwind, so everything allocated before the failure is in no Orphans and in no core state. It is unrecoverable and no current test can see it. The design’s answer — escalate to killing the isolate, because killing the process is how you reclaim state you cannot enumerate — is correct and is not yet built.

8.6 arm64 is measured for the core and unmeasured for the shell

The full suite *ran* on a GB10 Grace-Blackwell aarch64 host: **372 passed, 0 failed**, identical to x86_64. That is a real upgrade over a cross-compile check, and it covers the concurrency suites, whose memory-ordering behaviour is exactly what a cargo check cannot see.

What it does not establish, stated by the project so nobody over-reads it: `getconf PAGESIZE` was 4096 on that host, so the 16/64 KiB page-size rule — the one real pressure point — is still unexercised. The box was a container with no `/dev/kvm` and no `/dev/userfaultfd`, so **no L1/L2 OS-shell behaviour was exercised on ARM at all**. And there are zero VM-capable arm64 machines on the bench provider, so settling the remaining question by experiment requires hardware from somewhere else.

The honest summary is the project’s own: *the pure core is arm64-clean by measurement; the OS shell on arm64 is entirely unmeasured*.

8.7 The QEMU $\geq$ 10.2 floor is a distribution constraint

The design requires `memory_region_enable_lockless_io()`, which lands in QEMU 10.2. That buys a genuinely important thing — MMIO dispatch without the big QEMU lock, which is what makes the doorbell path viable — and upstream’s helper also sets the reentrancy guard in the same body, so a discipline the design planned to maintain is no longer ours.

The cost is that 10.2 is very new. Anyone deploying this needs a QEMU newer than what stable distributions ship, and the alternative previously considered — backporting roughly four lines onto 9.2 — is a maintenance burden with a hypervisor-integrity failure mode. The floor is the right call and it is a real adoption tax.

The facility inventory that priced the floor also **reversed two of its own design assumptions**, which is worth knowing before trusting the rest of it: the read-only-memslot region-lock fallback, priced in the design at 1.49 μ s as “a flags-only flip”, is *unreachable through QEMU* — QEMU’s flatten pass turns a readonly change into a delete-plus-add. And checkpoint-restart (`cpr-transfer`) is a lifecycle event the design had never considered.

8.8 The bench is a single serialised GPU

All hardware work happens on one rented GPU box, strictly serially: concurrent or mid-ioctl activity wedges the GPU. The C artifact needs a **fresh VM boot per clean run** — and, per §3, that discipline turns out to be a bug rather than a precaution. Two cargo builds on one box corrupted a mutation run through the shared `~/.cargo`, so the rule is one cargo per box.

The consequence for the reader: every hardware number in this document is $n = 1$ in the statistical sense and comes from one GPU model (GA106/GA107 class consumer Ampere) on one driver family. There is no second machine, no second GPU generation, and no continuous hardware testing.

8.9 The recurring failure mode: claims that were true *once*

This is the pattern the project keeps hitting, it has a name in the repository, and it is the single most useful thing for a reviewer to internalise.

Across three independent passes, roughly **twenty** instances were found of documentation asserting more than the code did — **several stating the literal opposite of their own code**. **Every one of them was true when written**. That is the point: this is not carelessness, it is entropy. [docs/design/testing_doctrine.md §6]

The instances are specific and they are not small:

- A design document named `memory_region_clear_global_locking()` as the mechanism for a decision. **That QEMU function had been deleted five years earlier**. The reference document written to price the floor now carries a `file:line` on every row for exactly this reason — and found two *more* live instances of the same decay while being written.
- A refactor plan cited a `userfaultfd` proof-of-concept file **that does not exist**. The mechanism was planned four times and never built; its demand-fault path is dead code that calls `stub_exit(139)` and never returns, so the fault address has been permanently zero since a commit early in the project's life. Nobody noticed because "planned" and "presumably done" read the same in a plan.
- **Two sections of one design document contradicted each other**, found only when the code between them was written.
- Two claims about the C's behaviour, cited from the C's own source comments, turned out to be **false when measured on the bench**. The rule extracted: citing a comment is citing a belief.
- A rustdoc described a default-to-GPU-0 guess *inside the one resolver whose entire discipline is "never guess"*. Another claimed "MISS=FAULT is preserved — never a silent skip" while its own body correctly skipped on both arms; the body was right and the sentence was wrong about the most-cited exception in the codebase.

The project's response is the right one: prefer a mechanism to a sentence. A rule stated only in prose has no reader. `HostHandle` carrying its `IsolateId` is the same rule with a compiler behind it. The house style for corrections is *strike visibly* — keep the original, struck, with the correction and its evidence — so a reader who remembers the old claim learns it was wrong rather than doubting their memory.

And the pattern is live right now, in this document's subject. Writing this paper found a dozen more instances in the Rust repository alone (§10). Every single one of them is a claim with *no gate behind it*. Every claim that *does* have a gate — the boundary greps, the ratchet at 37, the reached-count floor at 35, the "exactly one GPA call site" assertion — is mechanically true today. That is either a devastating footnote or the strongest possible evidence for the project's own thesis, and it is the latter.

8.10 Risks this document adds

- **Test code slightly exceeds implementation code, and the mocks are the only implementation of three ports.** The suite is therefore testing the core against a model of NVIDIA written by the same author as the core. `MockArch`'s deliberately non-NVIDIA encodings mitigate the *encoding* half of that; they do not mitigate the *semantics* half. If a belief about RM semantics is wrong, the suite will confirm it consistently.
- **The project is three days old and moves faster than its own documentation.** 93 commits in 72 hours, with several design documents rewritten mid-build. That is what produced eight stale claims in a repository that has an explicit doctrine against stale claims. It will produce more.
- `kayfabe-vmm-kvm` **defect #57 is a lock-invariant violation under race, and it is blocking by design.** That is the correct call. It is also the kind of defect that is cheap to live with and expensive to leave: an R1 violation that fires 1-in-6 under contention is exactly the shape that becomes a rare production hang.
- **The security model has been red-teamed at the logic layer only.** Memory safety, out-of-bounds and host-breakout are explicitly deferred to a post-L2 audit, on the sound argument that they are born at the `mmap/isolate/trap` surfaces and there is nothing to audit in pure logic. Those surfaces now partly exist (`kayfabe-linux-raw`, `kayfabe-vmm-kvm`), so the deferral is beginning to age.
- **The `Vmm` port's hypervisor-agnosticism is contracted and CI-gated but has exactly one adapter.** "A second backend costs one crate and zero trait changes" is a claim that only a second backend can settle.

9 How to read the code

9.1 Entry points

If you want to understand...	Read
the whole system, fastest	README.md, then ARCHITECTURE.md — but read §10 of this paper first, because both are stale in specific ways
what a “process” is and why	crates/kayfabe-core/src/project.rs — the pure derivation
the source of truth	crates/kayfabe-core/src/rmgraph.rs — the recounted graph and the parked-fact machinery
the transactional apply	crates/kayfabe-core/src/gpu.rs, Spine::apply (:1202)
the forwarding entry points	crates/kayfabe-fwd/src/lib.rs — start at plan_doorbell (:1140) and read the ring gate at :1162--1175
the concurrency design	crates/kayfabe-rt/src/lock.rs (ranks) then device.rs verb_op (:1035) — the whole plan/execute/commit shape is one function
the unsafe surface	ls crates/kayfabe-linux-raw/src/*_unsafe.rs — that is the complete list, by construction
what a test is supposed to look like	tests/tests/l1_mean.rs (the arbiter) and tests/tests/miss_taxonomy.rs (the sharpest single property)

9.2 Conventions

- *_unsafe.rs — any file using the unsafe keyword is named this way, and the gate greps whole words so writing *about* the rule (unsafe_code, _unsafe.rs) never trips it.
- plan_* / commit_* — the two halves of the verb seam. plan_ runs under locks and issues nothing; commit_ runs after re-acquisition and re-validates.
- route_* / exec_* — the route/act split (structural rule R4): resolve identity against device-wide state, then act on one process’s state.
- Test names carry their provenance: t12_, t13_, t14_ are C-bug regressions; cb* are the regression matrix rows; b1_–b6_ are security boundary cases; i1–i4 are the isolation properties; the marker *Mutation-gate kill* in a doc comment names the mutant a test exists to kill.

9.3 Which design documents to read, in order

1. docs/design/testing_doctrine.md (321 lines) — read this *first*, even before the architecture. It is the shortest path to understanding what the project believes evidence is.
2. docs/design/core_state_and_consolidation.md — the reviewed per-crate state and the L1 hand-off contract.
3. docs/design/core_security_threat_model.md — the attacker model and the I1–I4 properties.
4. docs/design/l1_concurrency.md §12 and docs/design/l1_os_shell.md §14 — the contact logs (the two documents run to 10 265 lines between them). **Read these rather than the summaries**; they are where the design was found to be wrong, which is the part a summary cannot carry.
5. docs/reference/ — measured ground truth, kept out of the design documents so a wrong fact is corrected in one place. Note the version caveat in rm_semantics_measured.md §0 before quoting any number from it.

10 Claims corrected while writing this paper

Each of these is a repository claim that this paper checked against the tree and found stale or wrong. They are listed because the list is more useful than any individual correction: it is a measurement of the drift rate described in §8.9. None is a defect in the code; all are defects in descriptions of the code.

Claim, and where	What the tree says
1 “handle_doorbell is the <i>ONLY</i> caller of RmBackend::ring_doorbell” — ARCHITECTURE.md:112, core_completeness_gate.md:119, core_state_and_consolidation.md:174, fwd/src/lib.rs:15	False as stated. ring_doorbell has exactly one call site and it is inside Worker::execute, two hops away. plan_doorbell is the sole constructor of VerbPlan::Doorbell, and it has <i>two</i> callers: handle_doorbell and SharedDevice::doorbell — and the L1 shell path, the one a real guest MMIO write takes, goes through the second and never enters handle_doorbell. <i>The safety property survives</i> because the gate is inside plan_doorbell; the cardinality claim does not. Figure 3 draws the box around plan_doorbell.

	Claim, and where	What the tree says
2	kayfabe-abi is a <i>"stub (shape only)"</i> — README.md:29,67,134, ARCHITECTURE.md:22,34,57, and its own Cargo.toml description	False. 7 455 lines of source, a working offline generator, 11 generated structs, a 13-method version-dispatch decode surface, 2 272 lines of oracle tests. It is one of the most finished crates in the repository.
3	<i>"the implementor is the L1 shell (kayfabe_rt::SharedDevice)"</i> — ARCHITECTURE.md:38--43, kayfabe-vmm/src/lib.rs:816, core/src/gpu.rs:1012	No impl Device for SharedDevice exists . The only two implementors are test types that wrap an Arc<SharedDevice> and forward. ARCHITECTURE.md:38 is honest that nothing implements it "yet"; the surrounding text reads as though the shell already does.
4	<i>"283 tests"</i> — README.md:85, ARCHITECTURE.md:156	515 measured on 2026-07-27.
5	<i>"four standing CI gates"</i> — README.md:87	Six jobs; eleven steps in the stable job alone. The list of four omits the VMM-vocabulary gate, the GPA-accessor gate, three unsafe-containment asserts, the KVM reached-count floor, the fuzz-compile job, the slow job and the mutation job.
6	<i>"TSan (28 threaded tests, 0 races)"</i> — README.md:88	28 was the first campaign. The four TSan targets today contain 65 #[test] functions. The "0 races" half is not disputed here — it was not re-run.
7	<i>"the two measured-slow tests"</i> — README.md:87,108	Four to six, depending on how you count; the g10_* capped pair joined later.
8	<i>"the conformance suite (23 files)"</i> — ARCHITECTURE.md:62,138	28 files in tests/tests/, plus 5 under crates/*/tests/.
9	core_mutation_gate.md's 91% threshold and its <i>Reproduce</i> block	Both describe the pre-2026-07-27 four-path scope. The document was last touched 26 hours before the scope change and does not mention it. The workflow itself marks the threshold pending re-derivation .
10	kayfabe-mmu::walker's own doc: <i>"Also here (skeleton this milestone): the GMMU walk algorithm"</i> — mmu/src/lib.rs:30	There is no algorithm: one trait, one enum, 41 lines, zero constructors, zero implementors. ARCHITECTURE.md correctly says "skeleton"; the crate's own doc overstates.
11	kayfabe-gsp's doc: RPC decode <i>"against kayfabe-abi's generated message layouts"</i>	kayfabe-gsp does not depend on kayfabe-abi. Nothing does.
12	<i>"Isolate/RmBackend/Vmm trait-only (mock-implemented)"</i> — ARCHITECTURE.md:31,56	True for Isolate/RmBackend. False for Vmm: crates/kayfabe-vmm-kvm/src/lib.rs:1142 is a real KVM adapter with a real memory plane.
13	<i>"~14 months of quirk capture"</i> in the C's architecture and ABI documents	Git: 2026-05-24 to 2026-07-24. Two months, 739 commits.
14	The framing <i>"7B LLM at 63 tok/s"</i> as a headline C result	63.4 is a Mode-1 A/B endpoint. The audited parity figure is 0.97× (66.0 guest vs 67.8 host) . The underlying milestone still records the pre-fix 20 tok/s and was never updated. All three numbers are real; they are different builds.

10.1 What this paper could not verify

- **Any claim requiring a build or a run.** No cargo command was executed while writing this. Test counts, mutation scores, TSan results, timings and the arm64 run are all read from commit messages, workflow files and design documents, and are attributed as such. The 515 figure in particular is a recorded measurement, not one taken here.
- **"TSan, 0 races."** Not re-run; only the target list and the test count in those targets were checked.
- **The relationship between the C's 2026-06-16 bare-metal result and the 2026-07-25 one-process-per-lifetime finding.** The memory note flags this itself: the finding was not cross-checked against the bare-metal box, and warns against assuming the C regressed.
- **Whether Mode 2 ever produced any display output.** No positive evidence was found, only the plan. Absence of evidence.

- **The #57 defect’s exact reproduction rate.** “~1 in 6 under race” is quoted from `crate_maturity_map.md`; it was not reproduced here.

A Measurement method

Every number in this paper that is not attributed to a document was produced by one of the following, run against `/workspace/nvkvm-rs` at `0b78102` and `/workspace/nvidia-gpu-passthrough` at `2899a74`. No compilation, no test execution and no network access was involved.

Quantity	Command
implementation lines	<code>find crates -path '*/src/*' -name '*.rs' xargs wc -l</code>
test lines	the same over tests, <code>crates/*/tests</code> , fuzz
per-crate lines	the same, per <code>crates/<name></code>
<code>#[test]</code> sites	<code>grep -rc '#[test]'</code> over workspace members, excluding the detached <code>crates/kayfabe-abi/gen</code> and the <code>trybuild ui fixtures</code>
unsafe relaxations	<code>grep -rhoE 'unsafe ({ fn impl trait })'</code> <code>crates/kayfabe-linux-raw/src/*_unsafe.rs wc -l</code> — the same expression the CI ratchet uses
dependency edges	every <code>Cargo.toml</code> read in full, cross-checked against <code>Cargo.lock</code>
commit counts and dates	<code>git rev-list --count HEAD</code> , <code>git log --format=%ci</code>
document staleness	<code>git log -1 --format=%ci</code> per file, compared against the commit that invalidated the claim

The five figures are TikZ, drawn from the measurements above and from a function-by-function read of the call chains they depict; they are vector, not traced from an image. Figure 2 in particular is generated from the manifest edge list rather than from any design document, which is why it disagrees with the hexagon picture about which crates are “ports”.

One closing note on how to use this paper. If you are here to find what is wrong with `kayfabe`, the highest-yield reading order is: §8.2 (the unbuilt critical path), §7’s final subsection (what the suite cannot prove), the two contact logs named in §9.3, and then the code — starting with `plan_doorbell` and `Spine::apply`, which between them contain most of the load-bearing decisions. Do not start with `README.md`; §10 lists six things wrong with it, and it will cost you an hour.